

Lucrarea nr. 2

Lucrul cu memoria grafică: adresare, prelucrări la nivel de bit

1. OBIECTIVELE LUCRĂRII

Lucrarea își propune prezentarea modalităților de accesare directă a memoriei video a calculatorului. Pentru aceasta sunt prezentate principalele funcții BIOS utilizate pentru accesarea directă a memoriei video, și elemente legate de lucrul cu acestea. De asemenea, sunt prezentate o serie de aplicații, unele rezolvate altele propuse ca tema.

2. BREVIAR TEORETIC

2.1 Funcții BIOS

BIOS-ul (Basic Input Output System) este un ansamblu de rutine ce oferă un set de servicii și funcții standard pe care orice sistem să fie capabil să le efectueze. Aceste rutine sunt rezidente în memoria ROM a calculatoarelor IBM-PC sau compatibile.

De fapt, BIOS-ul reprezintă o interfață între sistemul de operare sau programele utilizator și partea de hardware a calculatorului. Prin acest sistem de rutine se pot utiliza foarte ușor resursele hardware ale calculatorului: displayul, tastatura, imprimanta, căile seriale, discurile magnetice.

Conținutul rutinelor din BIOS diferă în funcție de tipul de calculator și nu pot fi rulate decât pe sistemul pentru care au fost proiectate, dar rezultatul execuției acestora este același indiferent de varianta constructivă. În acest fel calculatorul poate fi făcut mai performant folosind circuite mai puternice, cu păstrarea compatibilității cu variantele anterioare. De asemenea, utilizatorul are posibilitatea de a scrie propriile sale rutine prin care să controleze hardware-ul calculatorului. Aceasta implică, însă, cunoașterea în detaliu a circuitelor de intrare/ieșire cu care este dotat calculatorul.

Apelarea unei funcții BIOS se face printr-o instrucțiune de tip *întrerupere software*. Aceasta instrucțiune are mnemonica **INT i**, unde $i \in \{0, FF\}$. Zona de memorie corespunzătoare tabelului vectorilor de întrerupere începe de la adresa 00000h și cuprinde 1 Ko până la adresa 003FFh.

Exemple de funcții BIOS

- **INT 05h** → întrerupere folosită pentru a tipări conținutul ecranului la imprimantă. Aceasta întrerupere este apelată direct de întreruperea de la tastatură atunci când sunt apășate tastele <Shift> și <Print Screen>.
- **INT 08h** → întrerupere de la ceasul de timp real. Aceasta întrerupere este generată de aproximativ 18,2 ori pe secundă.
- **INT 09h** → întrerupere de la tastatură. Aceasta întrerupere este generată ori de câte ori se apăsă sau se eliberează o tastă.

- **INT 10h** → întrerupere video. Aceasta întrerupere oferă o serie de servicii pentru gestionarea ecranului.

Exista mai multe tipuri de servicii video. Pentru utilizarea lor se apelează întreruperea INT 10h, după ce, în prealabil, în registrul AH s-a încărcat codul serviciului dorit. În continuare se prezintă câteva servicii video.

- **Serviciul 00h** → selecția modului de lucru. Terminalul grafic poate lucra în două moduri: *alfanumeric* și *grafic*. Modulul de lucru *alfanumeric* sunt caracterizate de numărul de linii caracter (25), numărul de coloane caracter (40 sau 80), atribute de culoare și numărul de pagini. Pentru acest mod de lucru, un caracter este specificat prin codul ASCII și atributul de culoare, care conține trei elemente (culoare fond, culoare caracter, afișare continuă sau intermitentă - blinking). Modul de lucru *grafic* se caracterizează prin numărul de linii și coloane (de puncte de pe ecran – pixeli), precum și culoarea. Registrul AL va specifica modul de lucru selectat:
 - **00h** = alfanumeric 40 * 25, alb/negru
 - **01h** = alfanumeric 40 * 25, color (16)
 - **02h** = alfanumeric 80 * 25, alb/negru
 - **03h** = alfanumeric 80 * 25, color (16)
 - **04h** = grafic, 320 * 200, color (4)
 - **05h** = grafic, 320 * 200, alb/negru
 - **06h** = grafic, 640 * 200, alb/negru
 - **0Dh** = grafic, 320 * 200, color (16)
 - **0Eh** = grafic, 640 * 200, color (16)
 - **0Fh** = grafic, 640 * 350, alb/negru
 - **10h** = grafic, 640 * 350, color (16)
 - **11h** = grafic, 640 * 480, alb/negru
 - **12h** = grafic, 640 * 480, color (16)
 - **13h** = grafic, 320 * 200, color (256)
- **Serviciul 01h** → stabilește dimensiunea și forma cursorului, selectând liniile sale de început și de sfârșit. Pentru apelul serviciului se folosesc regiștrii:
 - AH** ← 01h
 - CH** ← numărul liniei de început a cursorului (0 - 1Fh, biții 0 - 4)
 - CL** ← numărul liniei de sfârșit a cursorului (0 - 1Fh, biții 0 - 4).
- **Serviciul 02h** → stabilește poziția cursorului pentru o pagina video specificată. Coltul stânga sus este specificat prin rândul 0h și coloana 0h, iar colțul dreapta jos prin rândul 18h (24) și coloana 4Fh (79). Pentru apelul serviciului se folosesc regiștrii:
 - AH** ← 02h
 - BH** ← numărul paginii video
 - DH** ← numărul rândului
 - DL** ← numărul coloanei.
- **Serviciul 03h** → returnează poziția și dimensiunea cursorului pentru o pagina video specificată. Pentru apelul serviciului se folosesc regiștrii:
 - AH** ← 03h
 - BH** ← numărul paginii video.
 Serviciul returnează următoarele:
 - CH** ← linia de început a cursorului
 - CL** ← linia de sfârșit a cursorului
 - DH** ← rândul pe care se afla cursorul

DL ← coloana în care se afla cursorul.

- **Serviciul 05h** → selectează pagina de afișare activă. Pentru apelul serviciului se folosesc registrii:

AH ← 05h

AL ← numărul paginii de afișare (de la 0 la 7).

- **Serviciul 06h** → realizează defilarea în sus a unei ferestre date, cu un anumit număr de rânduri. Pentru apelul serviciului se folosesc registrii:

AH ← 06h

AL ← numărul de rânduri de defilat

BH ← atributul de afișare pentru rândurile goale

CH ← numărul rândului pentru colțul stânga sus

CL ← numărul coloanei pentru colțul stânga sus

DH ← numărul rândului pentru colțul dreapta jos

DL ← numărul coloanei pentru colțul dreapta jos.

- **Serviciul 07h** → realizează defilarea în jos a unei ferestre date, cu un anumit număr de rânduri. Apelarea serviciului este similară cu cea a serviciului **06h**, cu deosebirea că în registrul **AH** se încarcă 07h.

- **Serviciul 08h** → returnează codul ASCII și atributul caracterului de la poziția curentă a cursorului, pentru o pagină video specificată. Pentru apelul serviciului se folosesc registrii:

AH ← 08h

BH ← numărul paginii video.

Serviciul returnează următoarele:

AH ← atributul caracterului

AL ← codul ASCII al caracterului.

- **Serviciul 09h** → afișează un caracter cu un anumit atribut de un număr specificat de ori. Afișarea se face de la poziția curentă a cursorului din pagina video specificată. Pentru apelul serviciului se folosesc registre:

AH ← 09h

AL ← codul ASCII al caracterului

BH ← numărul paginii video

BL ← atributul caracterului

CX ← numărul care specifică de câte ori va fi afișat caracterul.

Atributul unui caracter este definit astfel:

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

D2 D6	D1 D5	D0 D4	Culoare caracter Culoare fond
0	0	0	Negru
0	0	1	Albastru
0	1	0	Verde
0	1	1	Azuriu
1	0	0	Roșu
1	0	1	Purpuriu
1	1	0	Maro
1	1	1	Alb

D2 D1 D0 → culoarea caracterului
D3 → intensitatea culorii de caracter
D6 D5 D4 → culoarea fondului
D7 → clipire.

- **Serviciul 0Ch** → setează un pixel pe ecran la coordonatele specificate. Pentru apelul serviciului se folosesc regiștrii:

AH ← 0Ch

AL ← culoarea pixelului

CX ← coloana de pe ecran (coordonata x)

DX ← rândul de pe ecran (coordonata y)

Culoarea pixelului depinde de valoarea scrisă în memorie, astfel:

Valoare	Culoare
0	Black
1	Blue
2	Green
3	Cyan
4	Red
5	Magenta
6	Brown
7	Light Gray
8	Dark Gray
9	Light Blue
10	Light Green
11	Light Cyan
12	Light Red
13	Light Magenta
14	Yellow
15	White

Primele 16 culori VGA

Tabela ASCII.

Calculatoarele utilizează un set de caractere bazat pe 256 de simboluri: numere, cifre și caractere speciale. În tabelul de mai jos prezentăm tabela ASCII completă.

char	hex	char	hex	char	hex
NUL	00	*	2A	U	55
SOH	01	+	2B	V	56
STX	02	,	2C	W	57
ETX	03	-	2D	X	58
EOT	04	.	2E	Y	59
ENQ	05	/	2F	Z	5A
ACK	06	0	30	[	5B
BEL	07	1	31	\	5C
BS	08	2	32	]	5D
HT	09	3	33	^	5E
LF	0A	4	34	~	5F
VT	0B	5	35		60
FF	0C	6	36	a	61
CR	0D	7	37	b	62
SO	0E	8	38	c	63
SI	0F	9	39	d	64
DLE	10	:	3A	e	65
DC1	11	;	3B	f	66
DC2	12	<	3C	g	67
DC3	13	=	3D	h	68
DC4	14	>	3E	i	69
NAK	15	?	3F	j	6A
SYN	16	@	40	k	6B
ETB	17	A	41	l	6C
CAN	18	B	42	m	6D
EM	19	C	43	n	6E
SUB	1A	D	44	o	6F
ESC	1B	E	45	p	70
FS	1C	F	46	q	71
GS	1D	G	47	r	72
RS	1E	H	48	s	73
US	1F	I	49	t	74
SPACE	20	J	4A	u	75
!	21	K	4B	v	76
"	22	L	4C	w	77
#	23	M	4D	x	78
\$	24	N	4E	y	79
%	25	O	4F	z	7A
&	26	P	50	{	7B
'	27	Q	51		7C
(	28	R	52	}	7D
)	29	S	53	~	7E
		T	54	DEL	7F

3. PROBLEME REZOLVATE

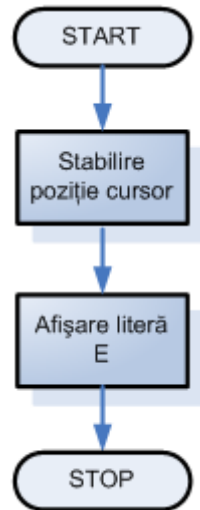
Problema 1

Realizați un program pentru tipărirea de zece ori a literei E pe fond albastru, culoare alba in centrul ecranului.

Soluție

Pentru rezolvarea problemei se vor folosi serviciile video BIOS apelate prin întreruperea **INT 10h**. Serviciile video folosite sunt:

- ↳ 02h pentru poziționarea cursorului în centrul ecranului (linia 0Ch, coloana 28h);
- ↳ 09h pentru scrierea literei E la poziția cursorului și având atributele specificate în enunțul problemei.



Programul în C (și limbaj asamblare) este următorul:

```

#include<iostream.h>
#include<conio.h>

void InitVideo()
{
    asm{
        mov ah,0x02    //permite stabilire pozitie cursor
        mov bh,0x00    //numarul paginii video
        mov dh,0x0c    //12 numarul randului
        mov dl,0x28    //40 numarul coloanei
        int 0x10
    }
}

void ScrieCaracter ()
{
    asm{
        mov ah,0x09    //afisare caractere cu attribute
        mov bh,0x00    //numarul paginii video
        mov al,0x45    //codul ASCII al lui A
        mov bl,0x1F    //atributul caracterului
        mov cx,0x0a    //specifica numarul de cate ori se va afisa caracterul
        int 0x10
    }
}

void main()
{
    clrscr();
    InitVideo();
    ScrieCaracter();
    getch();
}
  
```

Problema 2

Realizați un program pentru tipărirea aleatoare pe ecranul PC-ului a 65000 de pixeli. Tipărirea se va face în două moduri diferite:

- o dată prin apelarea întreruperilor BIOS,
- o dată prin scrierea directă a memoriei video.

Programul **demonstrează cu cât este mai rapidă scrierea directă în memoria video față de întreruperile BIOS.**

Soluție

Pentru rezolvarea problemei se vor folosi:

1. serviciile video BIOS apelate prin întreruperea **INT 10h**. Serviciile video folosite sunt:

- ↳ 00h pentru stabilire mod video de afișare;
- ↳ 0Ch pentru afișare pixel la poziția cursorului și având atributele specificate în enunțul problemei.

2. Scriere directă în memoria video:

- ↳ `VGA[y*SCREEN_WIDTH+x]=culoare;`

Programul în C (și limbaj asamblare) este următorul:

```
#include <stdio.h>
#include <stdlib.h>
#include <dos.h>
#include <time.h>
#include <conio.h>

#define VIDEO_INT          0x10    /* intreruperea BIOS pentru video */
#define WRITE_DOT          0x0C    /* func. BIOS pentru afisare pixel */
#define SET_MODE           0x00    /* func. BIOS pentru setare mod video */
#define VGA_256_COLOR_MODE 0x13    /* utilizare 256 culori */
#define TEXT_MODE          0x03    /* 80x25 mod text */

#define SCREEN_WIDTH       320     /* largime in pixeli in modul 0x13 */
#define SCREEN_HEIGHT      200     /* inaltime in pixels in modul 0x13 */
#define NUM_COLORS         256     /* numarul de culori in modul 0x13 */

typedef unsigned char  byte;
typedef unsigned short word;

byte *VGA=(byte *)0xA0000000L;    /* pointer la memoria video */

//Setare mod video
void set_mode(byte mode)
{
    asm{
        mov ah, SET_MODE
        mov al, mode
        int 0x10
    }
}

// Afisare lenta pixel
// Afisare pixel utilizand functia BIOS 0x0C (Write Dot)
void plot_pixel_slow(int x,int y,byte color)
{
    asm{
        mov ah, WRITE_DOT
```

```

        mov al, color
        mov cx, x
        mov dx, y
        int 0x10
    }
}

// Afisare rapida pixel
// Afisare pixel prin scriere directa in memoria video
void plot_pixel_fast(int x,int y,byte color)
{
    VGA[y*SCREEN_WIDTH+x]=color;
}

//Main: Afisam 64000 de pixeli in doua moduri diferite:
// 1. utilizand intreruperi BIOS
// 2. Scriind direct in memoria video
void main()
{
    int x,y,color;
    word i;
    clock_t start1,stop1,start2,stop2;
    float t1,t2;

    srand((unsigned) clock_t(&start1)); /* generatorul de numere pseudo-
aleatoare*/
    set_mode(VGA_256_COLOR_MODE); /* setare mod video */

    start1 = clock(); /* inregistrarea timpului de start */

    for(i=0;i<65000L;i++) /* afisare aleatoare a 65000 de pixeli */
    {
        x=rand()%SCREEN_WIDTH;

        y=rand()%SCREEN_HEIGHT;
        color=rand()%NUM_COLORS;
        plot_pixel_slow(x,y,color);
    }

    stop1 = clock(); /* calcularea timpului necesar */
    t1 = (stop1 - start1) / CLK_TCK;

    set_mode(VGA_256_COLOR_MODE); /* setare mod video din nou pentru
a sterge ecranul */

    start2=clock(); /* inregistrarea timpului de start */
    for(i=0;i<65000L;i++) /* afisare aleatoare a 65000 de pixeli */
    {
        x=rand()%SCREEN_WIDTH;
        y=rand()%SCREEN_HEIGHT;
        color=rand()%NUM_COLORS;
        plot_pixel_fast(x,y,color);
    }

    stop2=clock(); /* calcularea timpului necesar */
    t2 = (stop2 - start2) / CLK_TCK;

    set_mode(TEXT_MODE); /* trecere din mod video in mod text */

    /* afisare rezultate... */

```

```
printf("Afisare lenta pixel a durat %f secunde.\n",t1);
printf("Afisare rapida pixel a durat %f secunde.\n",t2);
if (t2 != 0)
    printf("Afisare rapida a fost de %f ori mai rapida.\n",t1/t2);
getch();
return;
}
```

Problema 3

Realizați un program pentru citirea primelor 10 locații ale memoriei video RAM. Modul de lucru al terminalului grafic va fi alfanumeric.

Soluție

Pentru rezolvarea problemei se vor folosi:

1. serviciile video BIOS apelate prin întreruperea **INT 10h**. Serviciile video folosite sunt:
 - ↳ 00h pentru stabilire mod video de afișare;
 - ↳ 03h pentru alfanumeric.
2. Adresa de start a memoriei video se selectează astfel:
 - ↳ 0xB800 pentru mod alfanumeric color
 - ↳ 0xA000 pentru mod grafic

Programul în C (și limbaj asamblare) este următorul:

```
#include <iostream.h>
#include <conio.h>

int CitesteMemVideo(int locatie)
{
    asm{
        mov ax,0xB800
        mov es,ax
        mov ax,locatie
        mov bx,ax
        mov ax,es:[bx]
        mov locatie,ax
    }
    return locatie;
}

void ScrieMemVideo(int locatie, int valoare)
{
    asm{
        mov ax,0xB800
        mov es,ax
        mov ax,locatie
        mov bx,ax
        mov ax,valoare
        mov es:[bx],ax
    }
}

void InitVideo()
{
    asm{
        mov ah,0x00
    }
}
```

```

    mov al,0x03 /* selectie mod alfanumeric */
    int 0x10
}

void main()
{
    clrscr();
    InitVideo();
    for(int i=0;i<10;i++)
    {
        getch();
        ScrieMemVideo(i,i);
        cout<<CitesteMemVideo(i)<<endl;
    }
    getch();
}

```

Problema 4

Scrieți un program care generează un efect de plasmă animată prin rotația paletelor de culori. Folosiți modul grafic 13h care permite o rezoluție de 320x200 și 256 de culori (accesibil prin funcții BIOS).

Soluție

Pentru rezolvarea problemei se va ține cont de următoarele aspecte:

↳ Intrarea în modul grafic 13h (specific VGA) se face astfel:

```

    mov ah,0
    mov al,13h
    int 10h (servicii video).

```

↳ Memoria video în acest mod grafic se poate trata ca o matrice de 320x200 octeți, fiecare corespunzător unui pixel. Modificarea unui pixel aflat pe ecran la coordonatele (x,y) se face scriind culoarea acestuia în memoria video: `MemorieVideo[y][x] = culoare;`

↳ Memoria video VGA cuprinde 64000 de octeți începând cu adresa A000:0000h. În C++ se poate accesa, spre exemplu, scriind o clasă `ECRAN_VGA` redefinindu-i operatorul de indexare astfel:

```

    unsigned char far *operator[](int linie)
    {
        return (unsigned char far *)MK_FP(0xA000,0)+320*linie;
    }

```

↳ Setarea paletelor de culori se poate face cu ajutorul întreruperii 10h, funcția 12h care permite modificarea conținutului regiștrilor DAC ce conțin codificarea culorilor:

```

    DAC (index) -> culoare RGB:
    les dx,paleta //tablou cu cel puțin nrIntrariPaleta struct. RGB
    mov bx, IndexDAC //prima culoare de modificat
    mov cx, nrIntrariPaleta //numărul de culori de modificat
    mov ah, 10h
    mov al, 12h
    int 10h

```

↳ O structură RGB conține trei octeți corespunzători componentelor roșu, verde și albastru (în această ordine) pentru fiecare culoare. Numai primii 6 biți din fiecare octet sunt semnificativi (maxim 64 de nuanțe din fiecare componentă, 262144 culori în total). Orice modificare în conținutul unui DAC se reflectă imediat pe ecran, ceea ce duce la perturbații vizuale dacă se modifică paleta de culori în timp ce se reîmprospătează conținutul ecranului. Este bine ca orice modificare să se facă atunci când spotul electronic care baleiază matricea

tubului cinescopic este stins ceea ce se poate afla verificând cuvântul de la portul 3Dah (dacă este egală cu 8, spotul este activ):

```

așteaptă:
    mov dx, 03Dah
    in al, dx
    test al, 08
    je asteapta

```

; acum se poate modifica paleta

↳ Accesarea acestui port este prea lentă folosind instrucțiuni în limbaj înalt de aceea se recomandă inserarea instrucțiunilor “asm” în program. În plus, o parte din regiștrii necesari modificării paletelor pot fi inițializați înainte de intrarea în bucla de așteptare. Pentru a obține o paletă cu degradeuri fine, se pot folosi 32 de nuanțe de albastru aranjate astfel: NEGRU ... ALBASTRU ... NEGRU (adică 64 de culori).

↳ În ceea ce privește crearea imaginii de bază pentru efectul de plasmă, folosiți o funcție care asociază fiecărui punct (x,y) o culoare ce depinde de distanța de până la origine. Limitând superior culoarea la 64 se vor obține cercuri concentrice cu centrul în origine de culoare periodică. La aceasta se pot adăuga variații ondulatorii cu ajutorul funcțiilor sinus și cosinus. În sfârșit, scala imaginii se poate modifica divizând coordonatele (x,y) cu factori oarecare.

Programul în C (și limbaj asamblare) este următorul:

```

#include<complex.h>
#include<conio.h>
#include<math.h>
#include<dos.h>
#include<iostream.h>
#include<stdlib.h>

#define scala 25
#define MaxCulori 64

typedef struct SRGB
{
    unsigned char r,g,b;
}RGB;

class ECRAN_VGA
{
private:
    unsigned char far*mem;
    union REGS r;
    SREGS s;
    unsigned char ModGrafic;
public:
    ECRAN_VGA()
    {
        MaxX = 320;
        MaxY = 200;
        mem = (unsigned char far*)MK_FP(0XA000,0);
    }
    unsigned char far* operator[](int linie)
    {return mem+320*(linie%200);}

    void IntrareModGrafic()
    {
        r.h.ah = 0xf; //memorez fostul mod grafic
    }

```

```

        int86(0x10, &r, &r);
        ModGrafic = r.h.al;
        r.x.ax = 0x13;
        int86(0x10, &r, &r);
    }

    void IesireModGrafic()
    {
        r.h.ah = 0; //ma intorc la fostul mod grafic
        r.h.al = ModGrafic;
        int86(0x10, &r, &r);
    }

    void SelecteazaPaleta (RGB far *paleta, int IndexStart, int
nrIntrari)
    {
        s.es = FP_SEG(paleta);
        r.x.bx = IndexStart;
        r.x.cx = nrIntrari;
        r.x.dx = FP_OFF(paleta);
        r.x.ax = 0x1012;
        int86x(0x10, &r, &r, &s);
    }

    int MaxX, MaxY;
}ecran; //inchid clasa

RGB pal[256], c; long i, x, y;
float v;
void main()
{
    union REGS r;
    SREGS s;
    ecran.IntrareModGrafic();
    for(i=0; i<MaxCulori/2; i++)
    {
        pal[i].r=0; pal[i].g=0; pal[i].b=i;
        pal[MaxCulori-i-1]=pal[i];
    }
    ecran.SelecteazaPaleta(pal, 0, MaxCulori);
    for(x=0; x<ecran.MaxX; x++)
        for(y=0; y<ecran.MaxY; y++)
        {
            v = sqrt(pow(float(x)/scala, 2)+pow(float(y)/scala, 2))+
                sin(float(x)/scala)+cos(float(y)/scala);
            v*=MaxCulori; //largesc domeniul de numere intregi
acoperit de v
            ecran[y][x]=int(abs(v)+0.5)%MaxCulori;
        }
    while(!kbhit())
    {
        c=pal[0];
        for(i=1; i<MaxCulori; i++) //rotesc paleta
            pal[i-1]=pal[i];
        pal[MaxCulori-1]=c;
        asm{
            mov ax, seg(pal);
            mov es, ax;
            mov bx, 0;

```

```

        mov cx, MaxCulori;
    }
asteapta:
    asm{
        mov dx, 0x03DA;
        in al, dx;
        test al, 0x08;
        je asteapta;
        mov dx, offset(pal);
        mov ax, 0x1012;
        int(0x10)
    }
    ecran.IesireModGrafic();
}

```

4. DESFĂȘURAREA LUCRĂRII

1. Se va citi breviarul teoretic. Se atrage atenția asupra faptului că aceste informații vor fi folosite și la desfășurarea altor lucrări de laborator.
2. Se vor studia exemplele prezentate pe parcursul acestui laborator urmărind efectul rulării acestora și buna lor execuție.
3. Se va încerca găsirea și a altor posibilități de soluționare a problemelor rezolvate.
4. Realizați un program pentru tipărirea pe ecranul PC-ului, în centrul acestuia, a textului “SISTEME DE PRELUCRARE GRAFICĂ” pe fond roșu cu litere galbene. Pentru rezolvarea problemei se vor folosi serviciile video BIOS apelate prin întreruperea **INT 10h**.
5. Realizați un program pentru tipărirea pe ecranul PC-ului a unei „mire” formată din benzi verticale de lățimi egale. Benzile vor avea următoarele culori: **Alb** (0FFh, 0FFh, 0FFh), **Galben** (0FFh, 0FFh, 00h), **Turcoaz** (40h, e0h, d0h), **Verde** (00h, 0FFh, 00h), **Mov** (0E0h, 0B0h, 0FFh), **Roșu** (0FFh, 00h, 00h), **Albastru** (00h, 00h, 0FF), **Negru** (00h, 00h, 00h). Pentru rezolvarea problemei se vor folosi serviciile video BIOS apelate prin întreruperea **INT 10h**.
6. Se va răspunde la următoarele întrebări:
 - a) Cum se poate modifica problema rezolvată 1 astfel încât tipărirea textului să se facă pe rândul 7 coloana 58?
 - b) Cum se poate modifica problema rezolvată 2 astfel încât trecerea de la întreruperile BIOS la scrierea directă a memoriei video să se facă numai la apăsarea unei taste?
 - c) Cum se poate modifica problema rezolvată 3 astfel încât citirea memoriei video să se facă în condițiile în care este setat modul de lucru grafic?